

Introduction	2
Objectifs clés de l'analyse	2
Outils Utilisés	2
1. Importation et compréhension des Données	3
2. Nettoyage et préparation des données	4
2.1 Standardisation des Attributs des Colonnes	4
2.2 Gestion des Valeurs Manquantes et des Incohérences	4
2.3 Analyse des Valeurs Uniques	5
Traitement	9
3. Recommandations	13
Conclusion	14

Introduction

Le présent rapport d'analyse a pour objectif d'examiner un jeu de données fourni dans le cadre d'une compétition visant à créer un assistant virtuel médical basé sur des **LLMs open-source**. L'assistant virtuel doit être capable d'aider les patients à comprendre leurs diagnostics, traitements et prescriptions, en utilisant un langage clair et adapté culturellement. Le jeu de données comprend des informations cliniques détaillées sur les patients, y compris des résumés de consultations médicales.

Ce rapport vise à fournir une analyse approfondie des données fournies, en mettant en lumière les aspects clés qui peuvent être exploités pour créer un assistant virtuel médical efficace. Les résultats de cette analyse serviront de base pour le développement de l'assistant virtuel, en identifiant les domaines où les informations peuvent être améliorées et simplifiées pour une meilleure compréhension par les patients.

Objectifs clés de l'analyse

1. **Comprendre la structure du jeu de données et ses caractéristiques clés :** Identifier les variables disponibles et leur qualité.
2. **Nettoyer les données :** Gérer les valeurs manquantes, les incohérences et les valeurs aberrantes.
3. **Analyser les performances des diagnostics :** Évaluer la distribution des diagnostics, les corrélations entre variables, et l'efficacité des résumés cliniques.
4. **Visualiser les tendances :** Utiliser des graphiques et des tableaux pour illustrer les tendances et les insights clés.
5. **Fournir des recommandations :** Proposer des actions concrètes basées sur les résultats de l'analyse.

Outils Utilisés

Pour cette analyse, nous avons utilisé Python dans environnement Google Collab Notebook, ainsi que les librairies suivantes :

- **Pandas** : pour la manipulation et le nettoyage des données
- **Numpy** : pour les calculs
- **Matplotlib** et **Seaborn** : pour la visualisation et les graphiques

1. Importation et compréhension des Données

Le jeu de données, **clinical_summary.csv**, contient des informations médicales relatives aux patients, y compris des résumés de consultations, des diagnostics, des traitements, et des prescriptions. Ce jeu de données a été fourni au format csv et servira de base pour notre processus d'analyse exploratoire.

Le dataset contient 50000 lignes et 10 colonnes

```
[3] 1 import pandas as pd
    2 import seaborn as sns
    3 import matplotlib.pyplot as plt
    4 import numpy as np
    5

[4] 1 df = pd.read_csv("/content/drive/MyDrive/clinical_summaries.csv")
    2 df.head()
```

	summary_id	patient_id	patient_age	patient_gender	diagnosis	body_temp_c	blood_pressure_systolic	heart_rate	summary_text	date_recorded
0	CS000001	P002538	61.0	M	COVID-19	34.416801	126.632921	92.0	COVID-19 PCR positive.	2024-08-14
1	CS000002	P002869	NaN	F	Typhoid	34.899780	102.679066	88.0	Abdominal pain, nausea.	2024-11-11
2	CS000003	NaN	20.0	F	Typhoid	32.243013	124.862591	94.0	Abdominal pain, nausea.	2024-12-07
3	CS000004	P004611	25.0	M	Anemia	39.070247	123.192566	71.0	Positive malaria test.	2025-06-11
4	CS000005	P006581	NaN	M	Anemia	34.377840	NaN	79.0	Elevated enzymes.	2025-04-14

Voici une description des colonnes principales :

Colonne	Description	Qualité
summary_id	Identifiant du résumé	~90% complet
patient_id	ID patient	~90% complet
patient_age	Âge du patient	~90% complet
patient_gender	Sexe du patient	~90% complet
diagnosis	Diagnostic principal	~90% complet
body_temp_c	Température corporelle (°C)	~90% complet
blood_pressure_systolic	Tension systolique	~90% complet
heart_rate	Fréquence cardiaque	~90% complet
summary_text	Résumé clinique textuel	~89% complet

date_recorded	Date de l'enregistrement	~89% complet
---------------	--------------------------	--------------

```
1 df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   summary_id            44939 non-null  object 
1   patient_id            44996 non-null  object 
2   patient_age           45031 non-null  float64
3   patient_gender        44948 non-null  object 
4   diagnosis             45037 non-null  object 
5   body_temp_c           44936 non-null  float64
6   blood_pressure_systolic 45037 non-null  float64
7   heart_rate            44996 non-null  float64
8   summary_text          44785 non-null  object 
9   date_recorded         44934 non-null  object 
dtypes: float64(4), object(6)
memory usage: 3.8+ MB
```

Le résultat de **df.info()** nous indique que notre jeu de données contient des variables numériques et catégorielles, ainsi que des **valeurs manquantes** dans toutes les colonnes.

2. Nettoyage et préparation des données

Cette section se concentre sur le nettoyage et la préparation des données pour une analyse approfondie. Le but est d'assurer la cohérence, l'exactitude et la fiabilité des données en standardisant les attributs des colonnes, en gérant les valeurs manquantes et les incohérences, en analysant les valeurs uniques, et en détectant les éventuelles valeurs aberrantes. Cette étape est cruciale pour garantir que les données sont prêtes pour l'analyse.

2.1 Standardisation des Attributs des Colonnes

Pour assurer l'uniformité et faciliter l'analyse, nous effectuerons les transformations suivantes :

- Changer le format de **date_recorded** en type date
- Mettre tous les diagnostics en minuscules

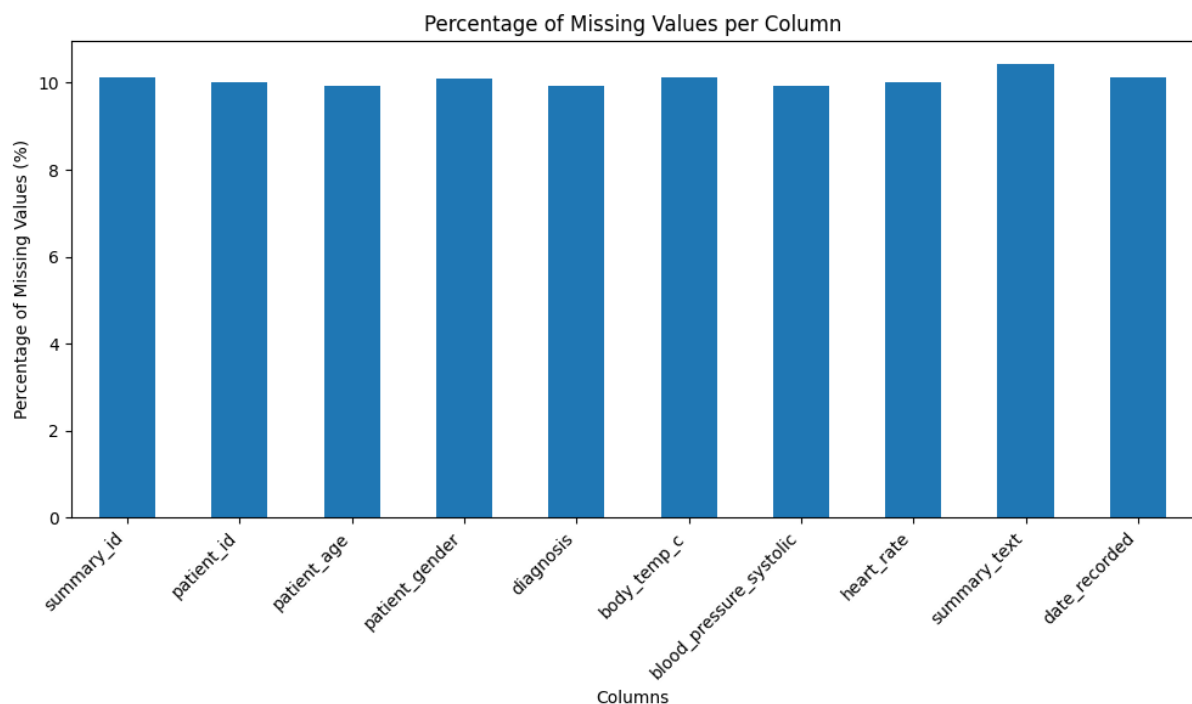
```
1
2 #Changer le format de date recorded en type date
3 df['date_recorded'] = pd.to_datetime(df['date_recorded'])
4
5 #Mettre tout les diagnostics en minuscules
6 df['diagnosis'] = df['diagnosis'].str.lower()
```

2.2 Gestion des Valeurs Manquantes et des Incohérences

Cette section se concentre sur la gestion des valeurs manquantes et des incohérences dans le jeu de données. Etant donnée que l'analyse initiale a montré que le jeu de données contenait des valeurs manquantes, nous allons réaliser une évaluation plus approfondie pour comprendre leurs sources, leur nombre, et

comment les gérer. De plus, nous examinerons les champs catégoriels et numériques pour détecter des valeurs incorrectes ou inattendues qui pourraient affecter notre analyse.

```
1 # Calculate the percentage of missing values for each column
2 missing_percentage = df.isnull().sum() / len(df) * 100
3
4 # Create a bar plot of missing value percentages
5 plt.figure(figsize=(10, 6))
6 missing_percentage.plot(kind='bar')
7 plt.title('Percentage of Missing Values per Column')
8 plt.xlabel('Columns')
9 plt.ylabel('Percentage of Missing Values (%)')
10 plt.xticks(rotation=45, ha='right')
11 plt.tight_layout()
12 plt.show()
```



On remarque que pour chaque colonne, il y'a environ **~10%** de valeurs manquantes.

Nous allons supprimer les valeurs manquantes dans ce jeu de données.

```
1 df.dropna(inplace=True)
```

2.3 Analyse des Valeurs Uniques

Nous examinerons la distribution des variables catégorielles pour comprendre la variété des valeurs uniques présentes dans chaque attribut. Cette étape aide à détecter des anomalies potentielles et des catégories en double.



```
1 print(f"Unique values in summary id: {df['patient_id'].unique()}")
2 print(f"Number of unique values in {'patient_id'}: {df['patient_id'].nunique()}")
```



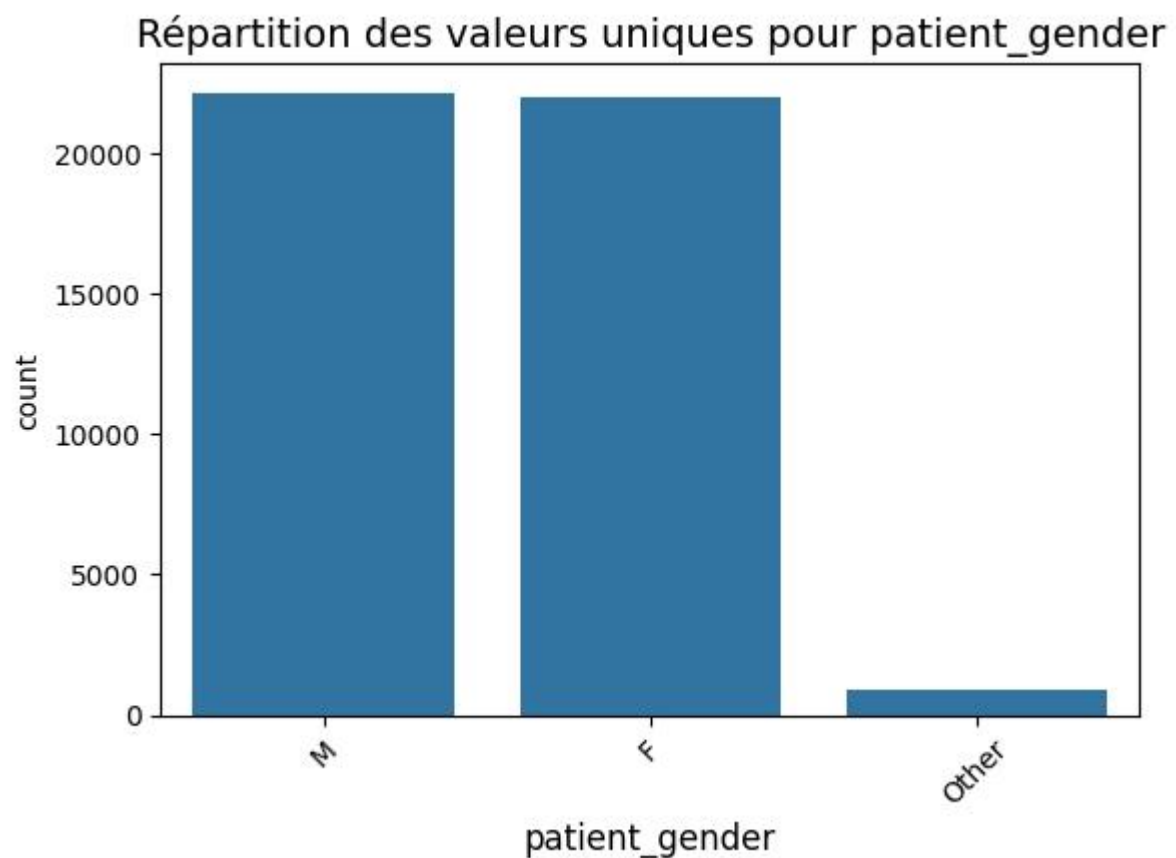
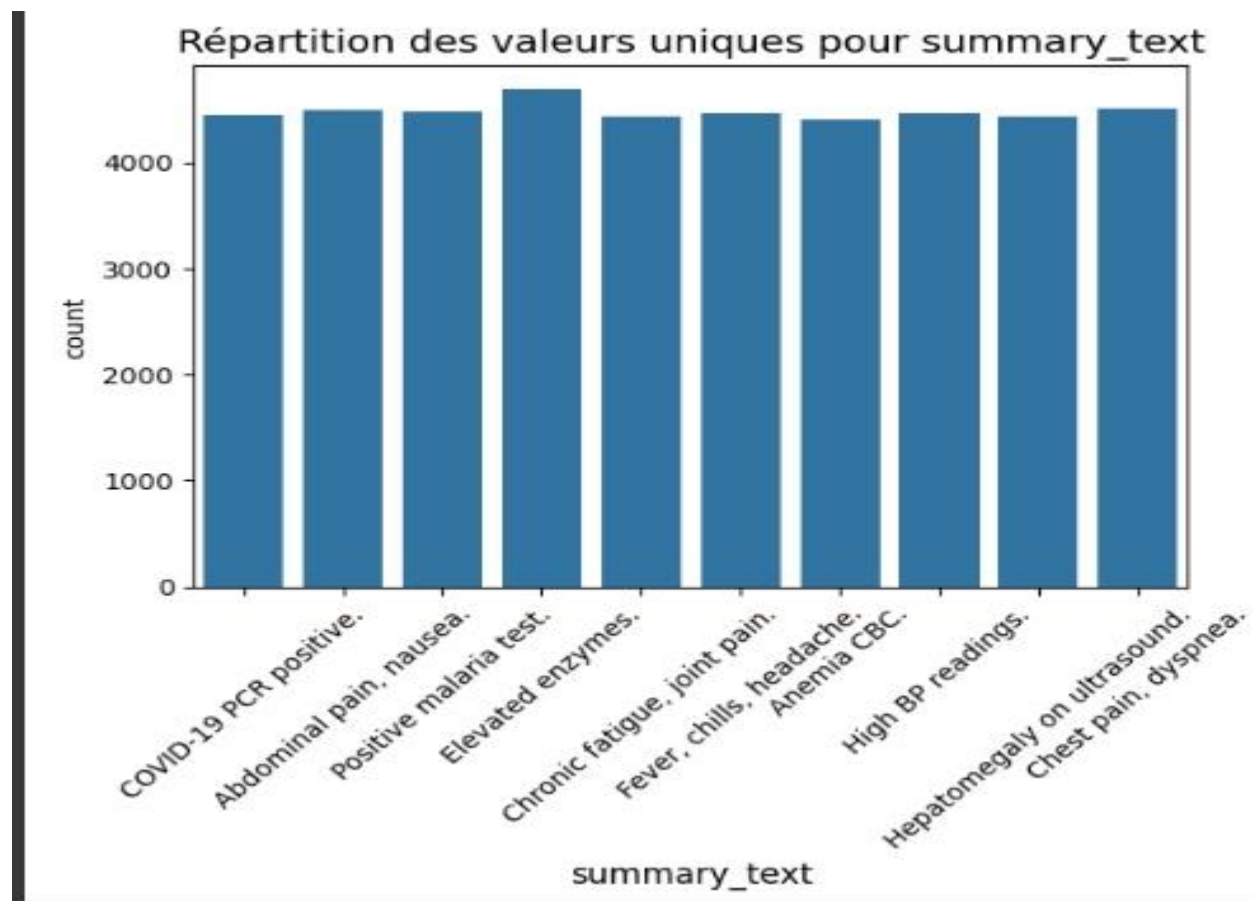
Unique values in summary id: ['P002538' 'P002869' nan ... 'P009208' 'P006222' 'P004253']
Number of unique values in patient_id: 9873

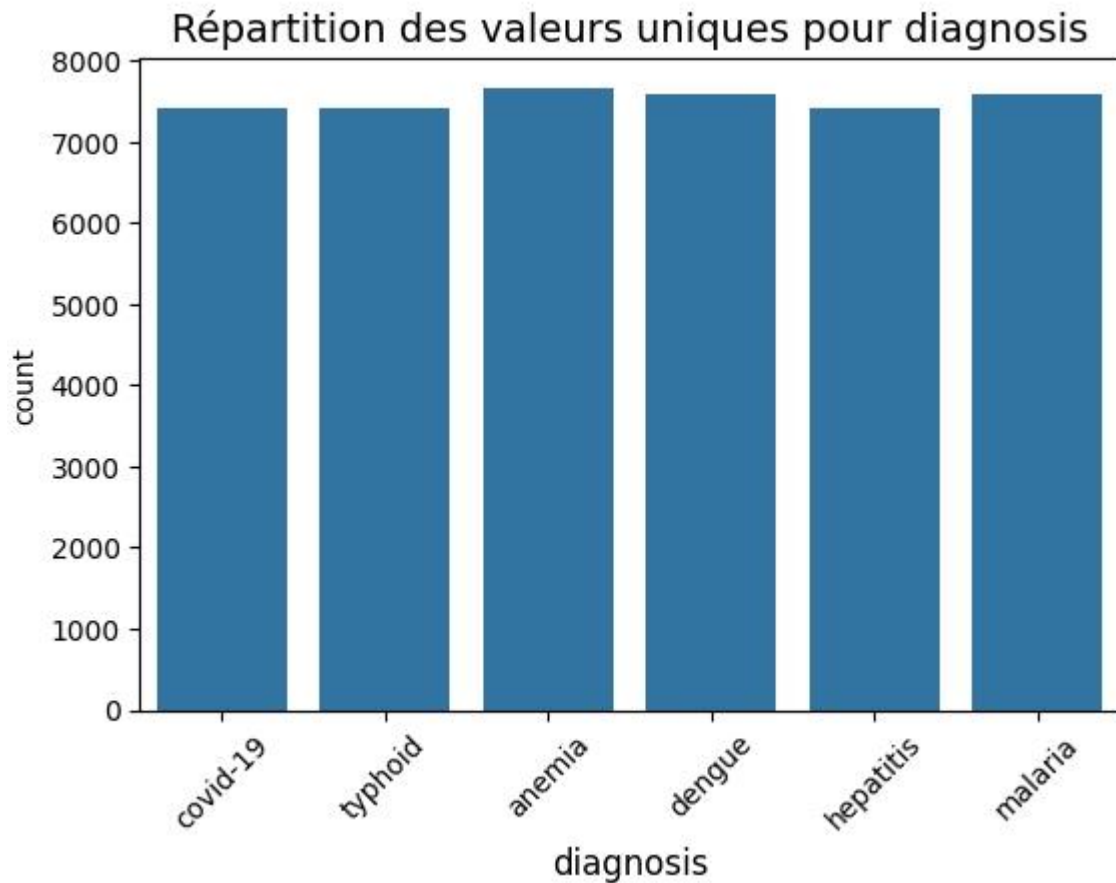
[16]

```
1 # Afficher les valeurs uniques pour chaque colonne catégorielle
2 print(f"Unique values in summary id: {df['summary_id'].unique()}")
3 print(f"Number of unique values in {'summary_id'}: {df['summary_id'].nunique()}")
4
```



Unique values in summary id: ['CS000001' 'CS000002' 'CS000003' ... 'CS049997' 'CS049998' 'CS049999']
Number of unique values in summary_id: 44939





- Il y'a **9873** patient_id unique, donc les patients se sont fait consulter plusieurs fois
- Il **44939** summary_id, on peut donc supposer que chaque consultation a un numéro unique
- Il y'a un équilibre de classe entre les sexes des patients, les *diagnostics* et les *summary_text*

Le nombre de patient_id par rapport au nombre de consultations suggère que les patients ont eu au moins une consultation par patient_id, ce qui est prouvé par :

```
[24] 1 #test_id = df['patient_id'].iloc[5]
      2 occ = {}
      3 for id in enumerate(df['patient_id']):
      4     df_id = df[df['patient_id'] == id[1]]
      5     occ[id] = len(df_id)
      6 df_occ = pd.DataFrame(occ.items(), columns=['id', 'occ'])
      7 df_occ['id2'] = df_occ['id'].apply(lambda x: x[1])
      8 df_occ.drop(columns=['id'], inplace=True)
      9
```

1 df_occ

	occ	id
0	2	P002538
1	6	P004611
2	1	P007031
3	2	P005627
4	4	P007282
...	...	...
17289	3	P007649
17290	4	P006008
17291	4	P008077
17292	4	P007257
17293	4	P004112

17294 rows x 2 columns

On remarque que pour certains patients, le sexe change entre deux consultations, ce qui pose une question ou un problème.

1 df_id

	summary_id	patient_id	patient_age	patient_gender	diagnosis	body_temp_c	blood_pressure_systolic	heart_rate	summary_text	date_recorded
8359	CS008360	P004112	20.0	F	anemia	37.838598	120.712217	76.0	High BP readings.	2025-04-09
28813	CS028814	P004112	70.0	F	malaria	36.975404	127.831728	74.0	Anemia CBC.	2025-06-19
32158	CS032159	P004112	57.0	F	typhoid	37.919717	115.084016	81.0	Abdominal pain, nausea.	2024-10-03
49998	CS049999	P004112	69.0	M	malaria	36.526933	130.637513	66.0	Anemia CBC.	2024-10-05

Traitement

1. On commence par filtrer le dataset pour ressortir les patients qui ont des sexes différents

```
[51] 1 inconsistent_gender_ids = df.groupby('patient_id')['patient_gender'].nunique()
     2 inconsistent_gender_ids = inconsistent_gender_ids[inconsistent_gender_ids > 1].index.tolist()
```

2. Affichage des patients de cette catégorie

```
1
2 selected_inconsistent_ids = inconsistent_gender_ids[:5]
3
4
5 for patient_id in selected_inconsistent_ids:
6     print(f"Records for Patient ID: {patient_id}")
7     display(df[df['patient_id'] == patient_id])
8     print("-" * 30) # Separator for better readability
```

Records for Patient ID: P000002

	summary_id	patient_id	patient_age	patient_gender	diagnosis	body_temp_c	blood_pressure_systolic	heart_rate	summary_text	date_recorded
2600	CS002601	P000002	79.0	M	hepatitis	39.489009	121.653523	79.0	COVID-19 PCR positive.	2024-08-21
7466	CS007467	P000002	59.0	F	malaria	40.170405	123.593980	76.0	COVID-19 PCR positive.	2024-11-02
27564	CS027565	P000002	83.0	F	dengue	37.942371	141.936887	70.0	Hepatomegaly on ultrasound.	2024-11-26
46352	CS046353	P000002	51.0	F	malaria	36.813077	125.031026	92.0	High BP readings.	2024-09-05

Records for Patient ID: P000004

	summary_id	patient_id	patient_age	patient_gender	diagnosis	body_temp_c	blood_pressure_systolic	heart_rate	summary_text	date_recorded
18760	CS018761	P000004	89.0	M	typhoid	37.614636	134.574113	85.0	COVID-19 PCR positive.	2025-06-04
37394	CS037395	P000004	59.0	Other	malaria	38.652440	118.718566	82.0	Positive malaria test.	2024-11-12

Records for Patient ID: P000006

3. Méthode 1 : Remplacer la colonne *patient_genre* par le sexe le plus **fréquent**

```
[53] 1 df_cleaned_method1 = df.copy()
      2
      3 for patient_id in inconsistent_gender_ids:
      4     patient_records = df_cleaned_method1[df_cleaned_method1['patient_id'] == patient_id]
      5     most_frequent_gender = patient_records['patient_gender'].mode()[0]
      6     df_cleaned_method1.loc[df_cleaned_method1['patient_id'] == patient_id, 'patient_gender'] = most_frequent_gender
      7
      8 print("Inconsistent gender values resolved by assigning the most frequent gender.")
```

Inconsistent gender values resolved by assigning the most frequent gender.

4. Méthode 2 : Remplacer la colonne *patient_id* par **Unknown** pour les patients ayant des sexes différents.

```
[56] 1 df_cleaned_method2 = df.copy()
      2
      3 for patient_id in inconsistent_gender_ids:
      4     df_cleaned_method2.loc[df_cleaned_method2['patient_id'] == patient_id, 'patient_gender'] = 'Unknown'
      5
      6 print("Inconsistent gender values addressed by assigning 'Unknown'.")
```

Inconsistent gender values addressed by assigning 'Unknown'.

Après application de la methode 1, on a

```
[60] 1 df_cleaned_method1[df_cleaned_method1['patient_id'] == 'P000007']
```

	summary_id	patient_id	patient_age	patient_gender	diagnosis	body_temp_c	blood_pressure_systolic	heart_rate	summary_text	date_recorded
12571	CS012572	P000007	64.0	F	dengue	34.517782	152.738030	64.0	Fever, chills, headache.	2025-04-02
26405	CS026406	P000007	35.0	F	malaria	37.877767	83.863038	76.0	Elevated enzymes.	2025-05-22
27879	CS027880	P000007	46.0	F	anemia	34.054520	104.035600	81.0	Hepatomegaly on ultrasound.	2024-11-25
49011	CS049012	P000007	44.0	F	dengue	37.252609	117.075335	75.0	Positive malaria test.	2024-07-24

Néanmoins, le choix de la méthode à appliquer dépendra des objectifs fixes et des méthodes utilisées.

De même, nous avons remarqué que certains patients avaient des âges différents entre deux consultations. Le choix de la méthode ici s'est porté sur la médiane des différents âges, car l'écart entre ceux-ci était très grand et la médiane est très peu sensible aux valeurs aberrantes. Voici le traitement

```
[ ] 1 # For each patient ID in the median_ages dictionary, replace the 'patient_age' in the DataFrame
2 for patient_id, median_age in median_ages.items():
3     df.loc[df['patient_id'] == patient_id, 'patient_age'] = median_age
4
5 print("Inconsistent patient ages replaced with median values.")
```

↩ Inconsistent patient ages replaced with median values.

Reasoning: Iterate through duplicate patient IDs, filter the DataFrame, check for age consistency, and calculate the median age for inconsistent entries.

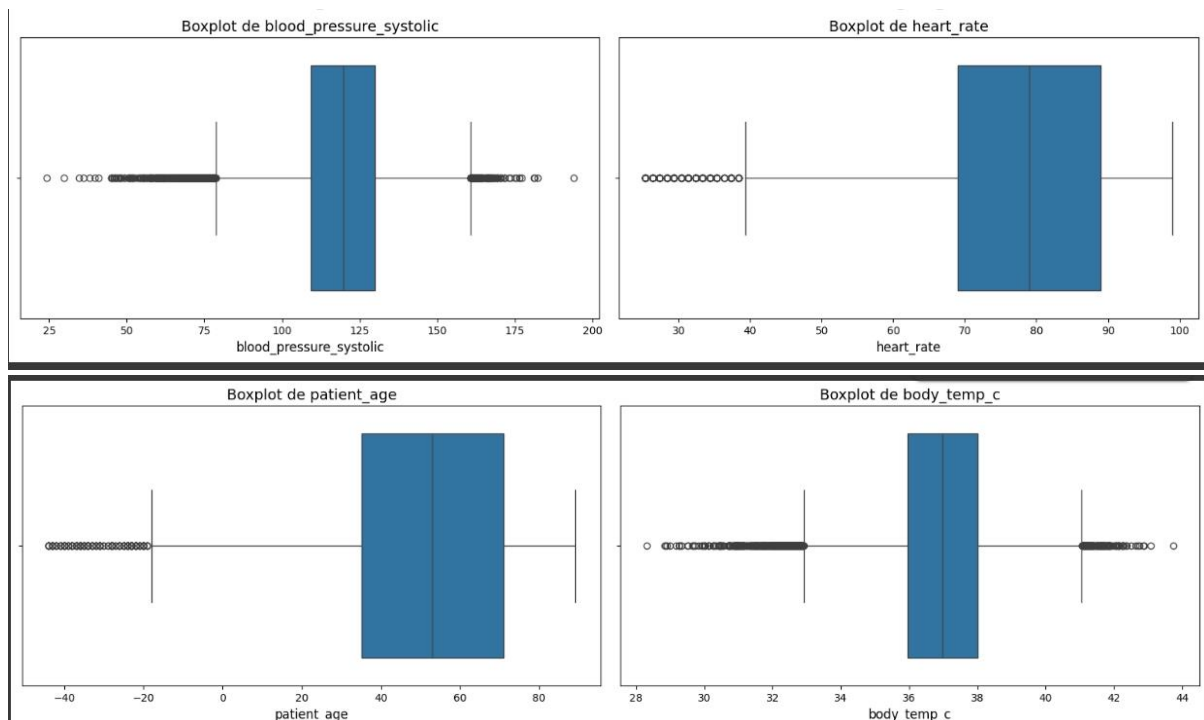
```
[ ] 1 # Dictionary to store median ages for patient IDs with inconsistent ages
2 median_ages = {}
3
4 # Iterate through the duplicate_patient_ids list
5 for patient_id in duplicate_patient_ids:
6     # Filter the DataFrame df to get all rows corresponding to that patient_id
7     patient_df = df[df['patient_id'] == patient_id]
8
9     # Check if all unique values in the 'patient_age' column for the filtered DataFrame are the same
10    unique_ages = patient_df['patient_age'].unique()
11
12    # If the number of unique values in 'patient_age' for the filtered DataFrame is greater than 1,
13    # calculate the median of the 'patient_age' for those rows and store it.
14    if len(unique_ages) > 1:
15        median_age = patient_df['patient_age'].median()
16        median_ages[patient_id] = median_age
17
18 print("Median ages calculated for patient IDs with inconsistent ages:", median_ages)
```

2.4 Détection des Valeurs Aberrantes

Identification des Valeurs Aberrantes

Nous utiliserons des boxplots pour visualiser les valeurs aberrantes dans les variables numériques. Les boxplots permettent de détecter facilement les valeurs extrêmes en affichant la distribution des données et en mettant en évidence les points qui se situent en dehors des whiskers (barbes).

```
1 # Liste des colonnes numériques
2
3 # Création des boxplots
4 plt.figure(figsize=(16, 18))
5 i = 1
6 for col in num_df:
7     plt.subplot(4, 2, i)
8     sns.boxplot(x=col, data=df)
9     plt.title(f'Boxplot de {col}', fontsize=14)
10    plt.xlabel(col, fontsize=12)
11    i += 1
12 plt.tight_layout()
13 plt.show()
```



Analyse des Boxplots

Les boxplots montrent la distribution des valeurs pour chaque variable numérique. Les points en dehors des whiskers sont considérés comme des valeurs aberrantes. Voici quelques observations :

- La colonne `patient_age` a des valeurs aberrantes (âge < -20)
- La colonne `blood_pressure_systolic` a des valeurs aberrantes (pression inférieure à 65 ou supérieure à 165)
- Les colonnes `heart_rate` et `body_temp_c` ont des valeurs aberrantes

Gestion des valeurs aberrantes

Dans cette partie, nous allons appliquer des techniques pour gérer les valeurs aberrantes.

→ Dans la colonne `patient_age`, on va remplacer les âges négatifs par leurs inverses.

```
[28] 1 df['patient_age'] = df['patient_age'].apply(lambda x: x if x>0 else -x)
```

```
1 df[df['patient_age']<0]['patient_age']
```

```
patient_age
```

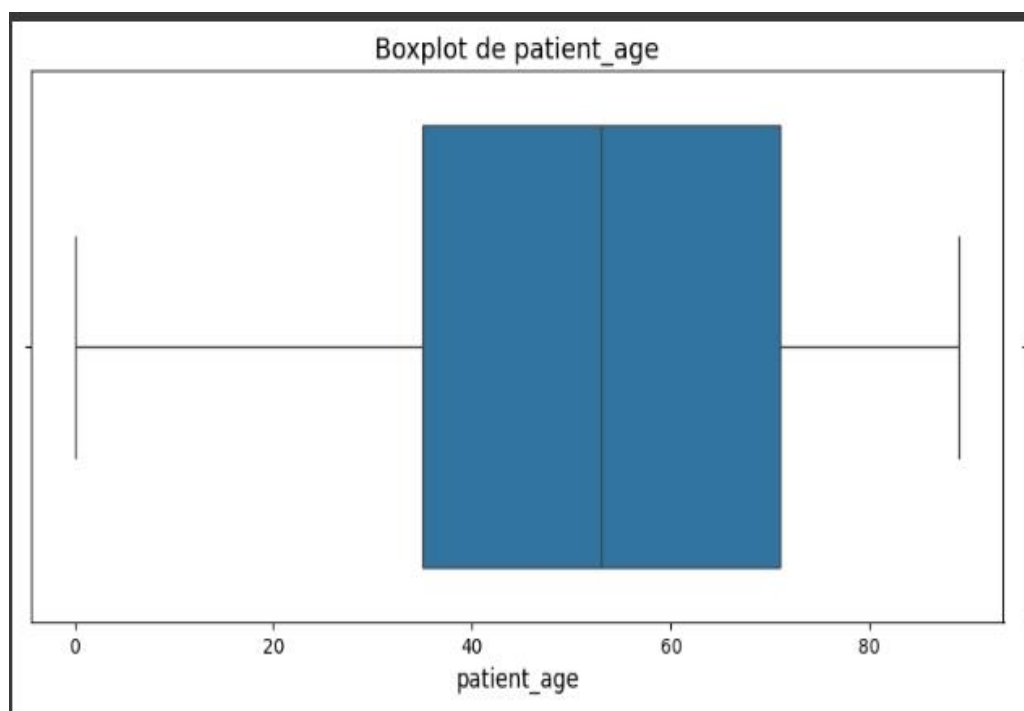
```
dtype: float64
```

→ Pour les colonnes 'body_temp_c', 'blood_pressure_systolic', et 'heart_rate', nous allons remplacer les valeurs aberrantes par **la médiane** de chaque colonne, car la médiane n'est pas sensible aux valeurs extrêmes.

```
1 # Replace outliers with the median using IQR method for numerical columns
2 numerical_cols = ['body_temp_c', 'blood_pressure_systolic', 'heart_rate']
3
4 for col in numerical_cols:
5     Q1 = df[col].quantile(0.25)
6     Q3 = df[col].quantile(0.75)
7     IQR = Q3 - Q1
8
9     lower_bound = Q1 - 1.5 * IQR
10    upper_bound = Q3 + 1.5 * IQR
11
12    # Replace outliers with the median
13    median_val = df[col].median()
14    df[col] = np.where(df[col] < lower_bound, median_val, df[col])
15    df[col] = np.where(df[col] > upper_bound, median_val, df[col])
16
17 print("Outliers replaced with median for numerical columns.")
```

Outliers replaced with median for numerical columns.

Après gestion, on obtient :



3. Recommandations

Cette section se concentre sur les recommandations issues de l'analyse du jeu de données. En plus des analyses précédentes, nous avons détecté des incohérences dans les données concernant le sexe des patients, certains ayant des sexes différents entre deux consultations. Pour résoudre ce problème, nous proposons deux méthodes de résolution : utiliser la valeur la plus fréquente ou remplacer par "Unknown". Les incohérences dans les données de sexe des patients peuvent affecter l'analyse et les

conclusions. Nous recommandons d'utiliser la méthode la plus appropriée en fonction des besoins spécifiques de l'analyse. Si la préservation des données originales est cruciale, la méthode de la valeur la plus fréquente est préférable. Si l'introduction de biais est un risque majeur, la méthode de remplacement par "Unknown" est plus appropriée. En tout cas, il est important de documenter et de justifier la méthode choisie pour assurer la transparence et la reproductibilité de l'analyse.

Conclusion

Cette analyse a permis de mieux comprendre les données médicales fournies et de détecter des incohérences potentielles. Les résultats de ces analyses fournissent des insights précieux pour créer un assistant virtuel médical capable d'aider les patients à comprendre leurs diagnostics, traitements et prescriptions de manière claire et adaptée culturellement. Les recommandations proposées pour résoudre les incohérences dans les données de sexe des patients peuvent être appliquées en fonction des besoins spécifiques de l'analyse.

En conclusion, l'analyse des données est un élément crucial pour garantir la qualité et la pertinence des informations fournies par l'assistant virtuel. En suivant les recommandations proposées, nous pouvons améliorer la fiabilité des données et ainsi fournir des explications plus précises et utiles aux patients.